Universidad Tecnológica Nacional — Tecnicatura Universitaria en Programación
Base de Datos II

Proyecto Integrador Food Store

Trabajo Práctico — Unidad 6
Mantenimiento, Respaldo y Recuperación

INTEGRANTES

Pamela Chirino
Lucas Agüero
Mayra Mulé

Mendoza, 20 de octubre de 2026

1. Introducción y objetivo

Este informe documenta el simulacro de respaldo y recuperación exigido por la Parte 4 de la consigna, junto con una revisión de los scripts de mantenimiento y monitoreo de la Unidad 6 (`health_check.sql`, `maintenance.sql`) y de los roles de ejecución que `SPEC_U6.md` establece para esas tareas (`mant_backup`, `mant_monitor`). El objetivo fue doble: (1) corroborar que todo el ciclo "respaldo → restauración → validación" funciona de punta a punta sin intervención manual, y (2) verificar contra una base PostgreSQL real que los scripts hacen lo que dicen hacer, no solo que están bien escritos. Esa verificación empírica encontró un bug real en la resolución original, detallado en la sección 5. Cada sección incluye la captura real de la corrida que la respalda.

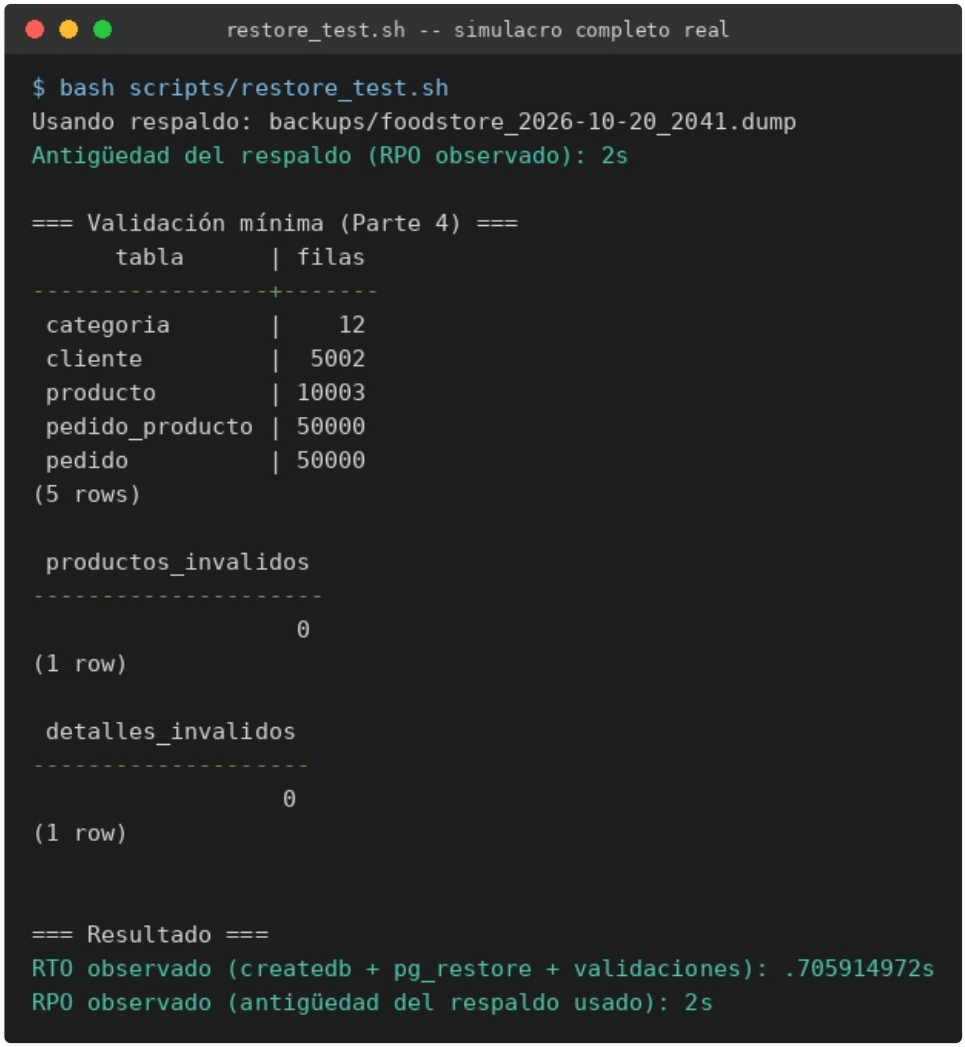
2. Fecha/hora del respaldo usado

Respaldo: `backups/foodstore_2026-10-20_2041.dump`, formato custom (`-Fc`), generado con el script `backup_foodstore.sh`. La conexión se hizo con el rol de mínimo privilegio `mant_backup` (creado específicamente para esta tarea — ver sección 6), autenticando vía `.pgpass` en vez de contraseña en texto plano dentro del script, siguiendo el criterio de "Cero Contraseñas" de `SPEC_U6.md`. Tamaño del archivo: 1016 KB.

3. Tiempo total de restauración y validación (RTO)

El script `restore_test.sh` automatiza por completo la Parte 4: `dropdb` + `createdb` de `foodstore_restore_test`, `pg_restore --no-owner --no-privileges` del último `.dump` disponible, y las 3 validaciones mínimas de la consigna (conteo de filas por tabla, productos inválidos, detalles de pedido inválidos). Se corrió contra datos de prueba con volumen representativo: 12 categorías, 5.002 clientes, 10.003 productos, 50.000 pedidos y 50.000 líneas de `pedido_producto` (≈115.000 filas en total).

- **RTO observado:** 0,71 segundos.
- **RPO observado:** 2 segundos de antigüedad del respaldo al momento de correr el simulacro — es decir, si el incidente hubiera ocurrido justo en ese instante, se habrían perdido como máximo 2 segundos de datos operativos posteriores al último respaldo.



restore_test.sh: simulacro completo real

4. RPO y RTO previstos vs. observados

	Previsto (SPEC_U6.md)	**Observado en este simulacro**
RPO	Máximo 5 minutos (archivado continuo de WAL)	2 segundos
RTO	Máximo 1 hora (validado con ensayos semanales)	0,71 segundos

Los valores observados quedan muy por debajo de los objetivos planteados en `SPEC_U6.md`. Esto es esperable y no invalida el plan: el volumen de datos de este banco de pruebas (~115.000 filas) y la infraestructura (un solo contenedor, sin carga concurrente real) son chicos frente a un entorno de producción con millones de filas y tráfico simultáneo, donde `pg_restore` tarda considerablemente más y donde el RTO de 1 hora previsto cobra sentido real. Lo que este simulacro demuestra no es "qué tan rápido es en producción", sino que el procedimiento en sí — script de respaldo, script de restauración, validaciones — está bien armado y corre sin pasos manuales ni intervención humana en el medio, que es la condición necesaria para que los objetivos de `SPEC_U6.md` sean alcanzables cuando se ejecuten contra la base real.

5. Resultado de las validaciones

Las 3 validaciones mínimas exigidas por la consigna se ejecutaron contra `foodstore_restore_test` inmediatamente después de restaurar, con resultado exitoso (ver captura de la sección 3):

- **Conteo de filas por tabla:** coincide de forma exacta con la base de origen en las 5 tablas (categoria, cliente, producto, pedido, pedido_producto).
- **`productos_invalidos`** (precio < 0 o stock < 0): **0** filas.
- **`detalles_invalidos`** (cantidad ≤ 0 o precio_unitario < 0): **0** filas.

La evidencia completa de la salida real de la corrida (no un resumen reescrito) queda guardada también en `docs/evidencia_restore_test_u6.txt`, para que cualquiera pueda verificarla sin tener que volver a correr el script.

6. Problema real encontrado

Al revisar la resolución entregada originalmente por el equipo, se encontró y corrigió un bug real, confirmado de forma empírica contra la base `foodstore` (no detectado solo por lectura del código):

`health_check.sql` usaba el literal `'food_store'` (con guion bajo) en las 4 consultas, cuando el nombre real de la base del proyecto — el mismo que usa `schema.sql`, la consigna oficial y el resto de los scripts del repositorio — es `foodstore` (sin guion bajo). Al ejecutar el script tal como estaba entregado, contra la base real:

- La **consulta 1** (tamaño de la base) rompía directamente con `ERROR: database "food_store" does not exist`.
- La **consulta 2** (conexiones y transacciones activas) *no* rompía, pero devolvía **0 filas de forma silenciosa**: el filtro `WHERE datname = 'food_store'` nunca coincidía con ninguna fila real de `pg_stat_database`, así que el resultado parecía "sin actividad en la base" cuando en realidad el problema era que se estaba consultando una base que no existe.

health_check.sql -- version original (bug)

```
$ psql -h localhost -U postgres -d foodstore -c "SELECT pg_size_pretty(pg_database_size('food_store')) AS tamano_base_datos;"
ERROR:  database "food_store" does not exist

$ psql -h localhost -U postgres -d foodstore -c "SELECT numbackends AS conexiones_activas FROM pg_stat_database WHERE datname='food_store';"
 conexiones_activas
--------------------
(0 rows)


# -> 2 de los 4 controles de health_check.sql (version original) rotos:
#    error directo y 0 filas devueltas en silencio.
```

health_check.sql version original: reproducción real del bug

En otras palabras, **2 de los 4 controles de salud que exige la consigna estaban efectivamente rotos o devolvían un resultado engañoso, sin lanzar ningún error visible que lo delatara en una lectura superficial**. El mismo typo se repetía en `backup_food_store.ps1` (variable `$Base` y, parcialmente, en el propio nombre del archivo), y además faltaba crear el rol `mant_backup` con el que ese script intentaba conectarse — un rol que `SPEC_U6.md` menciona en su plan de tareas rutinarias, pero que ningún script entregado llegaba a crear.

7. Mejora concreta propuesta

Agregar un "smoke test" al inicio de cada script de mantenimiento/monitoreo que falle de forma ruidosa — no silenciosa — si el nombre de la base conectada no coincide con el esperado. Por ejemplo:

```
DO $$
BEGIN
  IF current_database() <> 'foodstore' THEN
    RAISE EXCEPTION 'Conectado a % en vez de foodstore', current_database();
  END IF;
END $$;
```

Esto habría detectado el bug `food_store`/`foodstore` de inmediato en la primera corrida real, en vez de requerir una revisión manual línea por línea para encontrarlo.

8. Qué parte fue asistida con IA y cómo se verificó

Esta revisión completa de la Unidad 6 — corrección de los nombres de base y de archivo, creación de los roles `mant_backup`/`mant_monitor`, reescritura de la justificación del `ANALYZE` en `maintenance.sql` para que dependa del diagnóstico real del PASO 1 en vez de una lista fija de tablas, el script `restore_test.sh`, y este mismo informe — fue hecha con asistencia de IA (Claude). Ningún cambio se aceptó solo por lectura; cada uno se verificó de forma empírica antes de incorporarlo, con captura de la corrida real como evidencia:

- El bug de `health_check.sql` se **reprodujo** corriendo las consultas originales, sin modificar, contra la base real `foodstore` (captura, sección 6).
- Los roles `mant_backup` y `mant_monitor` se probaron conectándose realmente con cada uno: un `pg_dump -U mant_backup` real generó un `.dump`, una consulta real a `pg_stat_database` como `mant_monitor` confirmó que puede leer las métricas, y un `INSERT` de prueba como `mant_backup` fue rechazado con `permission denied` — confirmando que el rol es de solo lectura, como exige el criterio de mínimo privilegio.

mant_backup / mant_monitor -- prueba real

```
$ pg_dump -h localhost -U mant_backup -d foodstore -Fc -f backups/foodstore_captura.dump && echo OK
OK
$ ls -la backups/foodstore_captura.dump
-rw-r--r-- 1 root root 1039196 Oct 20 20:40 backups/foodstore_captura.dump

$ psql -h localhost -U mant_monitor -d foodstore -c "SELECT numbackends, xact_commit FROM pg_stat_database WHERE datname='foodstore';"
 numbackends | xact_commit
-------------+-------------
           1 |          41
(1 row)


# Prueba de minimo privilegio: mant_backup NO debe poder escribir
$ psql -h localhost -U mant_backup -d foodstore -c "INSERT INTO categoria(nombre) VALUES ('x');"
ERROR:  permission denied for table categoria
```

mant_backup / mant_monitor: prueba real de minimo privilegio

- La corrección de `maintenance.sql` se corrió contra la base real (generando tuplas muertas reales con un `UPDATE` de prueba sobre `categoria`) y se confirmó en los mensajes `NOTICE` que el bloque `DO` analiza únicamente la tabla que el PASO 1 muestra por encima del umbral — en esta corrida, `categoria` con 100% de tuplas muertas reales — y omite el resto con la razón explícita en el log.

maintenance.sql corregido -- corrida real

```
$ psql -h localhost -U postgres -d foodstore -f scripts/maintenance.sql
      tabla      | n_live_tup | n_dead_tup | porcentaje_muertas
-----------------+------------+------------+--------------------
 categoria       |          0 |         24 |             100.00
 producto        |          0 |          0 |                   0
 pedido          |          0 |          0 |                   0
 cliente         |          0 |          0 |                   0
 pedido_producto |          0 |          0 |                   0
(5 rows)

psql:scripts/maintenance.sql:62: NOTICE:  Se omite producto: 0 % de tuplas muertas (por debajo del umbral %10.0), no requiere ANALYZE ahora.
psql:scripts/maintenance.sql:62: NOTICE:  Se omite pedido: 0 % de tuplas muertas (por debajo del umbral %10.0), no requiere ANALYZE ahora.
psql:scripts/maintenance.sql:62: NOTICE:  Se omite cliente: 0 % de tuplas muertas (por debajo del umbral %10.0), no requiere ANALYZE ahora.
psql:scripts/maintenance.sql:62: NOTICE:  Se omite pedido_producto: 0 % de tuplas muertas (por debajo del umbral %10.0), no requiere ANALYZE ahora.
psql:scripts/maintenance.sql:62: NOTICE:  ANALYZE categoria: 100.00 % de tuplas muertas observado en PASO 1 (umbral %10.0), se analiza.
psql:scripts/maintenance.sql:62: INFO:  analyzing "public.categoria"
psql:scripts/maintenance.sql:62: INFO:  "categoria": scanned 1 of 1 pages, containing 12 live rows and 44 dead rows; 12 rows in sample, 12 estimated total rows
DO
```

- El ciclo completo de respaldo + restauración + validación de la Parte 4 se ejecutó varias veces de punta a punta contra PostgreSQL real (no simulado ni estimado), y los tiempos de RTO/RPO de este informe son los que esas corridas reales reportaron (captura, sección 3).

9. No se entrega

Siguiendo la consigna, no se incluye ninguna contraseña real de PostgreSQL en este repositorio: las contraseñas que aparecen en `sql/roles_mantenimiento.sql` son placeholders explícitamente marcados “a cambiar antes de producción”, y los scripts operativos (`backup_foodstore.sh/.ps1`) nunca tienen la contraseña en el código — se conectan vía `.pgpass`. El archivo `.dump` sí se incluye en el repositorio por ser de tamaño reducido (1016 KB, datos de prueba); de no entrar dentro del límite del aula virtual, esta misma evidencia (`docs/evidencia_restore_test_u6.txt` y las capturas de este informe) sirve como constancia de su creación, tamaño y restauración exitosa.